A Broken Heart 一个破碎的心

Or, getting a 100x speedup with one dumb line of code.
或者，用一行愚蠢的代码实现 100 倍速度提升。

January 31, 2026 • 7 min read

2026 年 1 月 31 日 • 阅读时长 7 分钟

You always know it's a good bug when your first reaction is, "How could this even happen?"

你总能知道这是一个好 Bug，因为你的第一反应是，"这怎么可能发生？"

The other day, I was refining the dashboard of a web app we're working on – as you do – and I noticed it was taking *forever* to load. Like, it had been loading in a single second, but now it was taking ten seconds. Something fishy was going on.

前几天，我正在优化我们正在开发的一个网络应用的仪表盘——就像平时那样——突然发现它加载得慢得离谱。之前它只需要一秒钟，但现在却要十秒钟。肯定出什么问题了。

Naturally, I blamed React.

我自然是归咎于 React。

I mean, sure, in a modern web app there are many potential causes of a performance problem: third-party JavaScript, overburdened servers, bloated assets, missing database indexes – a list as long as your arm. But decades of building for the web told me that this was a frontend problem. I could just smell it. The page looked janky while loading. And despite being the least-bad approach for web frontends today, the React ecosystem is lousy with ways for a codebase to get tangled, slow, and fishy.

我意思是，当然，在现代化的网络应用中，性能问题的原因有很多：第三方 JavaScript、服务器过载、臃肿的资产、缺失的数据库索引——数不胜数。但几十年来为网络构建的经验告诉我，这是一个前端问题。我直觉上就能感觉到。页面在加载时看起来很卡顿。而且尽管对于当今的网络前端来说是最不糟糕的做法，但 React 生态系统中充满了让代码库变得混乱、缓慢和诡异的方法。

So to prove my theory, I explained to Claude[1] that the dashboard was loading slowly, that it surely had some React problems, and to analyze it and rank them from most to least serious. And sure enough, Claude found a bunch of React fishiness – unnecessary re-renders, missing memoizations. We still weren't on React Compiler, which I hadn't realized. So I had Claude do a first pass on the easiest and most serious React issues, and…

所以为了证明我的理论，我向 Claude [1] 解释说仪表盘加载缓慢，肯定存在一些 React 问题，并要求他分析这些问题并按严重程度从高到低排序。果然，Claude 发现了一堆 React 的怪异之处——不必要的重新渲染、缺失的 memoization。我们仍然没有使用 React Compiler，这我之前没有意识到。所以我让 Claude 先处理最简单且最严重的 React 问题，然后……

It made almost no difference? Maybe it wasn't React after all.

几乎没什么区别？也许根本不是 React 的问题。

So, I rolled up my sleeves, and started investigating properly.

于是，我卷起袖子，开始认真调查。

1. Maybe the server really is slow? A little, but it's not blocking the frontend.

   服务器真的慢吗？有一点，但它并没有阻塞前端。

2. Was the problem in all browsers? No. It was somehow specific to Safari?

   问题出在所有浏览器上吗？没有。似乎是 Safari 特有的？

3. Ah, it must be third-party JavaScript then. Intercom? No. PostHog? No.

   啊，那一定是第三方 JavaScript 了。Intercom？不是。PostHog？不是。

4. Okay, let's really dig in to the performance timeline.

   好吧，让我们深入挖掘性能时间线。

Now, the Safari performance inspector has diverged from the Chromium one over the years, and has gotten (on this page at least) rather flaky. But it painted a pretty clear picture: It wasn't spending 7+ seconds parsing JavaScript, calculating styles, or loading from the network. It was using 94% of an M1 Max CPU on… Layout?

现在，Safari 性能检查器多年来已经与 Chromium 的有所分化，而且（至少在这个页面上）相当不稳定。但它描绘出了清晰的画面：它没有花费 7 秒以上解析 JavaScript、计算样式或从网络加载。它使用了 94%的 M1 Max CPU 在……布局上？

Digging into the details, it showed multiple Layout passes taking more than 1600ms each. For reference, that's roughly 100x slower than it should be, so something was seriously wrong with how our page was being laid out. Flexbox can get a little slow, but not *this* slow.

深入细节后，发现多个布局过程每个都耗时超过 1600 毫秒。作为参考，这大约是预期速度的 100 倍，因此我们的页面布局方式存在严重问题。Flexbox 可能会有些慢，但不会这么慢。

Time to tear things apart
是时候拆解问题了

At that point, I reached for an age-old tool that has gotten more useful in the modern age: binary search. That is, you explain the symptom to your coding agent. Then you have it repeatedly remove stuff from your code that might be causing the problem, and see if that fixes it. When you find *something* that fixes it, you iteratively re-add everything until you have a minimal change that indicates the underlying issue, and thus a workaround.

那时，我使用了古老的工具，在现代却变得更有用：二分查找。也就是说，你向你的编程代理描述症状。然后让它反复从你的代码中移除可能导致问题的部分，看看是否解决了问题。当你找到能解决问题的部分时，你迭代地重新添加所有内容，直到找到一个最小化变化的部分，表明潜在问题，从而找到一个解决方案。

This is especially fast if the agent can see the problem itself, but I didn't have a command-line Safari perf analysis tool on hand. Still, after only 10 minutes of telling Claude whether a given change did or didn't fix the issue, coaching it through how to think about what we'd just learned with each step, we'd found the culprit!

如果代理能看到问题本身，这将特别快。但我手头没有命令行 Safari 性能分析工具。尽管如此，在仅仅 10 分钟内告诉 Claude 某个变化是否解决了问题，指导它如何思考每一步学到的内容，我们找到了罪魁祸首！

A heart emoji. ❤️

一个心形表情符号。❤️

If we removed the emoji in our Send Feedback button (which I'd recently added), then Safari could lay the page out in 2 milliseconds. If we re-added it, the page took 1600ms for each layout, of which there were multiple.

如果我们移除了我们发送反馈按钮中的表情符号（这是我最近添加的），那么 Safari 可以在 2 毫秒内布局页面。如果我们再添加上它，每个布局需要 1600 毫秒，而且有多个布局。

Now, I like to use emoji in early prototype interfaces. They're trivial to add, and load faster than images. Right? A single character of a font should not take 100x longer to render (or, according to Safari, "Layout") than the rest of a dynamic React web app. Seems like I'd hit a Safari bug.

现在，我想在早期原型界面中使用 emoji。它们添加起来很简单，而且加载比图片快。对吧？一个字体的单个字符渲染时间不应该比整个动态 React Web 应用的其余部分慢 100 倍（或者，根据 Safari 的说法，"布局"）。似乎我遇到了一个 Safari 的 bug。

This is generally the "okay now I need a drink" moment in a Weird Bug Hunt. But it was still before noon, so I went to get another coffee instead.

这通常是在"怪异虫害大搜捕"中的"好吧，我现在需要喝杯咖啡"的时刻。但那时还没到中午，所以我决定再去拿一杯咖啡。

When you find something that looks to be a bug in the browser, you want to submit that bug. To submit a bug, though, you can't just attach your whole project. "Hey if you run this whole production app, Safari has a problem." You're meant to produce a minimal reproduction case: "Here's a simple file that triggers the issue. Load it and see." Even better, this means you'll fully understand the problem, and can likely

find a better workaround than “never use emoji in this app again”.

当你发现浏览器中似乎存在一个 bug 时，你想提交这个 bug。然而，要提交 bug，你不能只是附加你的整个项目。“如果你运行这个整个生产应用，Safari 有问题。”你应当制作一个最小的复现案例：“这是一个触发问题的简单文件。加载它并查看。”甚至更好，这意味着你将完全理解问题，并且可能找到比“永远不再在这个应用中使用表情符号”更好的解决方案。

Making a minimal repro sample is a huge barrier for submitters, though, since fully boiling a full-fledged app containing proprietary stuff down to the minimal repro case is a ton of tedium.

制作一个最小的可重现示例对提交者来说是一个巨大的障碍，因为将一个包含专有内容的全功能应用程序简化为最小的可重现情况是一项繁琐的工作。

Or, it used to be a ton of tedium! Related to their bug-isolating capabilities, coding agents are also particularly well-suited to producing minimal test cases. They can edit a lot of code at once, and it usually doesn’t take much creativity – you’re just iteratively removing as much as you can without making the bug stop triggering.

或者，以前这可是件非常繁琐的事情！与他们的 bug 隔离能力相关，编码代理也特别适合生成最小化测试用例。它们可以一次性编辑大量代码，而且通常不需要太多创造力——你只是不断地移除尽可能多的内容，直到 bug 不再触发。

So, before long, I had a very simple repro case for the Safari team. And, in looking at the minimal repro code, it became very clear what the real culprit was. On my Mac, Safari 26.2 takes 1600ms to “Layout” the following HTML.

所以，没过多久，我就为 Safari 团队准备了一个非常简单的复现案例。在查看最简复现代码时，我非常清楚地发现了真正的罪魁祸首。在我的 Mac 上，Safari 26.2 处理以下 HTML 需要 1600 毫秒才能完成“布局”。

```
<!DOCTYPE html>
<html>
<head>
  <link href="https://fonts.googleapis.com/css2?family=Noto+Color+Emoji" rel="stylesheet">
  <style>
    body { font-family: "Noto Color Emoji"; }
  </style>
</head>
<body>
  💔
</body>
</html>
```

Curse you, **Noto Color Emoji**.

诅咒你，Noto Color Emoji。

Fonts can have colors now!
字体现在可以有颜色了！

Traditionally, fonts were just shapes. It would display in black, or white, or whatever color you chose. But in 2008, Apple shipped Apple Color Emoji in iPhone OS 2.2, and brought it to Mac OS X in 2011. Emoji's rise in popularity led to demand for fonts that have intrinsic color.

传统上，字体只是形状。它会以黑色、白色或你选择的任何颜色显示。但在 2008 年，苹果在 iPhone OS 2.2 中推出了 Apple Color Emoji，并在 2011 年将其带到 Mac OS X。表情符号的流行导致了对具有内在颜色的字体的需求。

Originally, Apple's Color Emoji were basically a hack. They just stuck PNG images in a font, which was neither standardized nor resolution-independent. Outside a certain size range, they looked like butt. This led to four competing color font standards (from Apple, Mozilla,

Google, and Microsoft) all being submitted to OpenType 1.7. According to Wikipedia, Microsoft and Apple added support for these different approaches, and that's that.

最初，苹果的彩色表情符号基本上是一种变通方法。他们只是将 PNG 图像嵌入字体中，这既没有标准化，也不是分辨率无关的。在一定的尺寸范围之外，它们看起来就像一团糟。这导致了四个竞争的彩色字体标准（来自苹果、Mozilla、谷歌和微软）都被提交给 OpenType 1.7。根据维基百科的说法，微软和苹果为这些不同的方法添加了支持，事情就这样了。

But that – as it so often is – was not that.

但那——就像它经常的那样——并不是那样。

You see, Noto Color Emoji is a Google font that is helpful in that it gives you consistent emoji rendering across platforms. We'd included it earlier to get decent emoji rendering on Linux (where we do some HTML-to-video rendering in the cloud, a technique that sounds horrifying but can be pretty useful). However, the font relies on COLRv1, a spec Google advises will make your apps load faster because it results in smaller emoji than bitmaps – and can fall back to supplying SVG for other browsers.[2]

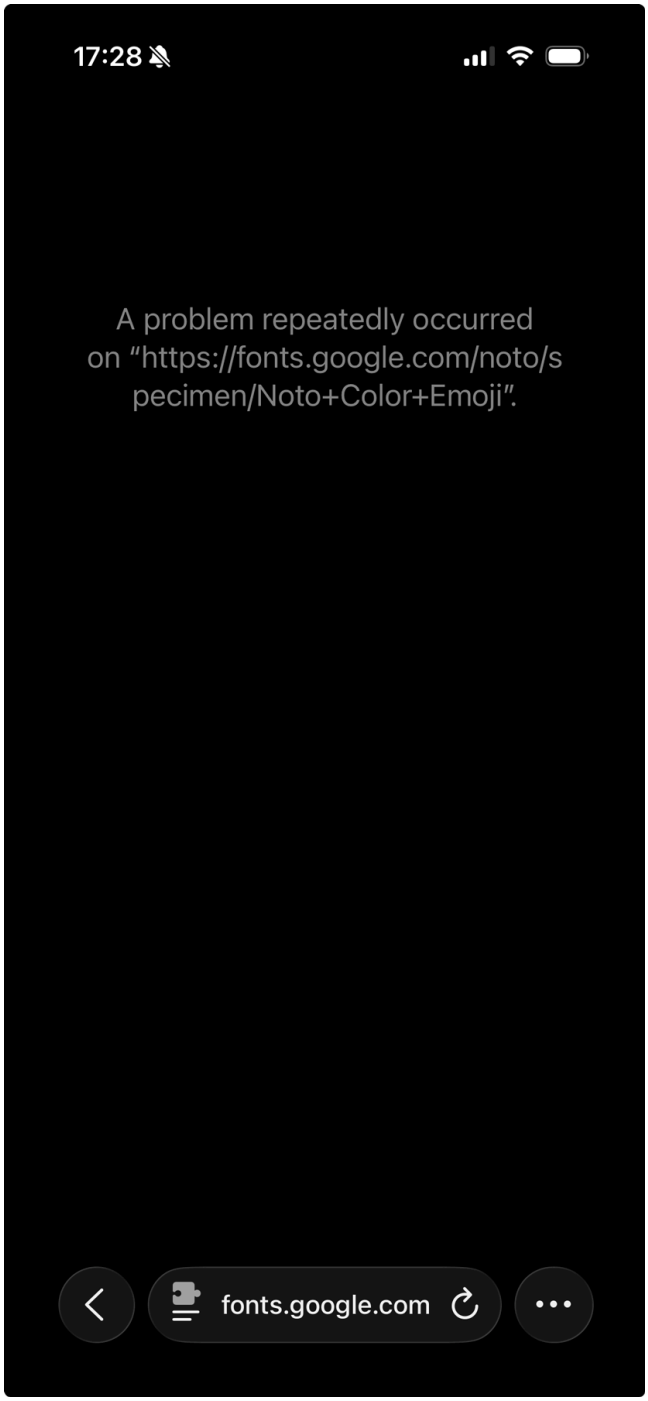
你看，Noto Color Emoji 是一款谷歌字体，它有助于在各个平台上提供一致的 emoji 渲染效果。我们之前加入它是为了在 Linux 上获得不错的 emoji 渲染（我们在云端进行一些 HTML-to-video 渲染，这种技术听起来很可怕，但实际上相当有用）。然而，这款字体依赖于 COLRv1 规范，谷歌建议该规范能让你的应用加载更快，因为它能生成比位图更小的 emoji——并且可以回退为向其他浏览器提供 SVG。[2]

"Other browsers", in this case, is Safari. And, I guess, "falling back to SVG" means spending 16000ms of "Layout" for a single character. If you'd like to see what this looks like scaled up, you can try loading the Google Fonts page that attempts to showcase all of the Noto Color

Emoji glyphs on iPhone. (As of iOS 26.2, it goes poorly.)

“其他浏览器”，在这种情况下是指 Safari。而且，我猜“回退到 SVG”意味着一个字符需要花费 1600 毫秒的“布局”时间。如果你想知道这种效果放大后的样子，可以尝试加载那个试图在 iPhone 上展示所有 Noto Color Emoji 字体的 Google Fonts 页面。（截至 iOS 26.2 版本，效果不太好。）

After I mentioned the bug in a Slack, Daniel Jalkut filed it in the Safari bug tracker, and Simon Fraser on the webkit team has already commented, noting the slowness seems to be within CoreSVG. Chances are this will get fixed!



在我提到这个 bug 后，Daniel Jalkut 在 Slack 上提交了它到 Safari 的 bug 追踪器，WebKit 团队的 Simon Fraser 已经评论了，指出慢速似乎在 CoreSVG 内部。很可能这个问题会被修复！

In the meantime, I'd like to contribute this humble finding to the search corpus: don't use Noto Color Emoji on Apple platforms – list "Apple Color Emoji" first. At least, until the bug is fixed and the resulting Safari release is widespread.[3]

与此同时，我想将这个卑微的发现贡献给搜索语料库：在苹果平台上不要使用 Noto Color Emoji——首先列出"Apple Color Emoji"。至少，在问题得到修复并且由此产生的 Safari 版本得到广泛传播之前。[3]

I'd also like to come clean on a little secret. As profoundly helpful as Claude was in debugging this – I surely fixed this problem 10x faster than I would have without it – I must admit, it was Claude that tipped us off to the existence of Noto Color Emoji in the first place. I suspect that, without the coding agent, we would have solved the Linux emoji problem in a more boring way (using an icon library) and not ended up with a weird slow emoji implementation.

我也想坦白一个小秘密。尽管 Claude 在调试这个过程中非常有帮助——没有它我肯定要慢 10 倍才能解决这个问题——但我必须承认，是 Claude 首先让我们发现了 Noto Color Emoji 的存在。我怀疑如果没有编程代理，我们可能会用一种更无聊的方式（使用图标库）解决 Linux 表情符号问题，而不会得到一个奇怪的慢速表情符号实现。

It seems more true every month: these coding agents are very much like a power saw. Profoundly useful, and proportionately dangerous.

每个月似乎都更加真切：这些编程代理非常像电锯。极其有用，也相应地危险。

So, I suppose, here's to Claude. The cause of – and solution to – all of startups' problems.

所以，我想，这是为克劳德干杯。创业公司所有问题的起因和解决方案。

---

1. As of publication, my default tool is Claude 4.5 Opus in Cursor or Claude Code. ↩

   截至目前，我的默认工具是 Cursor 中的 Claude 4.5 Opus 或 Claude Code。 ↩

2. The COLRv1 spec is ostensibly for supporting a variety of new font techniques like gradients, palettes, nested glyphs, and the like, but so far it seems to be mostly used for emoji. ↩

   COLRv1 规范表面上是为了支持渐变、调色板、嵌套字形等新字体技术，但目前似乎主要被用于表情符号。 ↩

3. *Addendum, Feb 11:* A couple folks from Google reached out informally about this bug. Dominik Röttsches noted that 🧺 is one of the hardest glyphs to render, but in my tests it doesn't seem to trigger this bug at all – ❤️ and 🤯 take 1600ms to lay out, but 🧺 and 🫠 take 0.2ms, implying it's a glyph-specific problem.

   补充说明，2 月 11 日：有几位来自谷歌的人非正式地联系过我关于这个 bug。Dominik Röttsches 指出🧺是最难渲染的符号之一，但在我的测试中它似乎完全不会触发这个 bug——❤️ 和🤯 的布局需要 1600 毫秒，而🧺 和🫠 只需要 0.2 毫秒，这表明这是一个特定于符号的问题。

   This meshes with Tom Roggero's note that the font can be patched to improve performance. I'll post another update if I hear that the font has been patched, which would surely be faster than a new version of Safari needing to propagate! ↩

   这与 Tom Roggero 的注释相符，即可以通过补丁来改进字体性能。如果听说字体已经被补丁修复，我会再发布另一个更新，这肯定比需要传播的新版 Safari 要快！ ↩

Next in the **Large Language Models** series: Maggie Appleton on Gas Town and Coding Agent Orchestration →

大型语言模型系列接下来：Maggie Appleton 谈 Gas Town 和编码代理编排→

Latest in the series **Bug Hunts**.

最新系列 Bug Hunts。

Latest in the series **Best Of**.

最新系列 Best Of。

Liked this? Follow along to see what's next.

喜欢这个？继续关注，看看接下来会发生什么。

© Allen Pike. 👋🏻 Feel free to contact me.

© Allen Pike. 👋🏻 欢迎联系我。